Field Hours

CSCE 616 Introduction to Hardware Design Verification

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A plan, not a waveform

You do not tape out a block because a few directed vectors looked right. You write a plan that names the behaviors, the stimulus that will reach them, and the checker that will judge them. A 4-bit adder has 256 input pairs if you count only the two operands, and 512 if you also toggle carry-in. The plan says which of those you will hit and which you will argue from a smaller set.

You separate three mechanisms that people collapse into the word test. Stimulus drives pins. A checker compares the pins to a rule, every clock, not only at the end. Coverage records which behaviors of the plan actually occurred. A simulation of 1000 cycles with an empty checker is not a passing test. It is a waveform you happened to save.

A passing log is the last box, and only if the first three were written down.

1. Name the behaviors

2. Drive the pins

3. Check every cycle

4. Record the bins

Corners of a 4-bit adder

1. Two 4-bit operands and a carry-in give 512 combinations. You do not type all of them on the first day.

2. You direct the corners: 0 plus 0, 15 plus 15, and 15 plus 15 plus carry-in. That last sum is 31, so the low nibble is 15 and carry-out is 1.

3. You cover carry-out with 2 bins. A plan that never drives carry-out 1 leaves a bin empty even when every nibble you tried was correct.

Problem 1. How many input combinations does a 4-bit adder have if you include carry-in, and why is a green waveform of one sum not a finished plan?

Answer. 512, because 16 times 16 times 2 is 512. One waveform shows one combination. The plan still needs a checker and a record of which behaviors, including both carry-out bins, actually occurred.

Problem 2. You ran 1000 cycles and saved the waveform, but you wrote no checker. Did the block pass?

Answer. No. A checker compares pins to a rule. An empty checker cannot pass or fail. Coverage of the plan was not recorded either.

Lecture 2 · 90 minutes

A directed testbench

You build a directed testbench around a clock and a reset. For the first 2 cycles you hold reset, then you release it and drive one input vector per cycle from a list you wrote. A 4-bit counter test might drive enable for 5 cycles and expect the count to move 0, 1, 2, 3, 4. You sample outputs on the clock edge the design uses, and you fail the test on the first mismatch.

You keep expected values next to the stimulus, not in your head. The mechanism is a cycle-by-cycle compare against that list, which catches a bug at cycle 3 instead of letting it poison cycle 4 and cycle 5. Directed tests are how you lock a known corner: reset, the all-ones input, a back-to-back pair. They do not scale to every legal interleaving, and you should not pretend a list of 20 vectors did.

Expected values sit beside the vectors. The first mismatch ends the test.

Hold reset two cycles -> Drive one listed vector -> Sample on the edge -> Fail on first mismatch

Enable a counter for 5 cycles

1. Reset is held for 2 cycles. The count must read 0 on the edge where reset releases.

2. Enable stays high for 5 cycles. You expect 1, then 2, then 3, then 4, then 5, each on the active edge.

3. If the third enabled cycle shows 4, you fail at that cycle. You do not wait to see whether the fifth cycle lands on 5 by luck.

Problem 1. Reset is held 2 cycles and enable is then high for 5 cycles, from a count of 0. What do you expect on the fifth enabled edge?

Answer. 5. The count moves 1, 2, 3, 4, 5 across the five enabled cycles. You fail at the first edge that disagrees, not at the end of the list.

Problem 2. Why is a list of 20 directed vectors not a complete check of every legal interleaving?

Answer. The list only locks the corners you wrote down. Interleavings you did not list are untested. Directed stimulus does not grow to cover them by itself.

Lecture 3 · 90 minutes

Assertions on the protocol

You attach assertions to the design so the check runs even when the stimulus came from someone else. An immediate assertion tests a Boolean on the spot, such as a one-hot grant: in any cycle the 4 grant bits contain at most one 1. A concurrent assertion talks about time. The property that a request is granted within 3 cycles fails if cycle t has a request and cycles t through t+2 have no grant.

You write the assertion next to the protocol, not inside one test. Then all 50 directed tests and every later random

test inherit the same check. A disabled assertion is not a waiver. If you turn it off to make a log quiet, you have deleted the checker. Keep the failure, and fix the design or the property you misstated.

Same pins, two kinds of check. Zero grants can pass one and fail the other.

Immediate

One Boolean this cycle

At most one of 4 grants

No window of cycles

Concurrent

A request starts a window

Grant must arrive in 3

Time is part of the check

A request that waits too long

1. At cycle 10 the request is 1 and the 4 grant bits are 0000. A window of 3 cycles starts.

2. Cycles 10, 11, and 12 stay at grant 0000. The concurrent property fails at cycle 12.

3. The immediate one-hot check still passes on 0000, because zero grants is not two grants. The two checks are not substitutes.

Problem 1. Grant bits are 0011 in a cycle. Does the immediate one-hot assertion pass?

Answer. No. Two of the 4 bits are 1, and the check allows at most one. The failure is this cycle; it does not wait for a later grant.

Problem 2. A request at cycle t is granted at cycle t+3. The bound is 3 cycles, counting t through t+2. Does the property pass?

Answer. No. Cycles t, t+1, and t+2 have no grant, so the window of 3 misses. A grant at t+3 is one cycle late.

Lecture 4 · 90 minutes

Constraints and seeds

You replace a hand-written list with a constraint solver that draws legal inputs. A constraint is a rule the draw must obey, such as length between 1 and 8, or opcode not equal to a reserved code. The solver picks one solution at random from that set. On a bus with 4 opcodes and a 3-bit length, the legal space is 4 times 8, which is 32, until a cross constraint deletes pairs.

You still need a seed. The same seed and the same constraints reproduce the same 100 draws, which is how you debug a failure. A different seed explores a different path through the 32 legal pairs. Constrained random finds interleavings you did not list. It will not hit a behavior your constraints made impossible, so a bug in the constraint is a hole in the stimulus, not a quiet success.

The solver only draws inside the rules. A seed makes the draw replayable.

1. Write the constraints

2. Draw one solution

3. Drive that draw

4. Save the seed

Thirty-two legal pairs, then twenty-five

1. Opcode is in {0, 1, 2, 3} and length is in 0 through 7. That is 4 times 8 = 32 legal transactions.

2. Seed 7 produces a sequence. A failure on draw 12 is rerun with seed 7, and draw 12 is the same transaction.

3. A further rule says opcode 3 implies length 0, which removes 7 pairs. The space is 32 minus 7 = 25. Opcode 3 with length 5 never appears.

Problem 1. Opcode has 4 values and length has 8. No cross constraint. How many legal transactions are there?

Answer. 32. The solver multiplies 4 by 8. A draw outside that set violates a constraint and must not be driven.

Problem 2. Seed 7 failed on draw 12. You change the seed and cannot reproduce the bug. What did you lose?

Answer. The replay. The same seed and the same constraints reproduce the same draws. A new seed is a different path through the legal pairs, not a rerun of draw 12.

Lecture 5 · 90 minutes

Bins and crosses

You measure what happened, not how long you ran. A coverpoint is a value you sample, and a bin is a bucket of that value. An opcode coverpoint with 4 bins, one per opcode, is hit for a bin only when a sampled transaction carries that opcode. Running 1000 cycles that all use opcode 0 leaves 3 bins empty. The test is not done.

You add a cross when the bug lives in a pair. Opcode crossed with a 2-value direction has 4 times 2 = 8 bins. Hitting every opcode and both directions in separate tests can still miss the pair you care about, such as opcode 3 with direction 1. Coverage of 6 out of 8 is 75 percent, and the missing 2 bins are the plan for the next constraint, not a rounding error.

A cross is not the sum of two coverpoints. Empty pairs stay empty until something hits them.

Coverpoint: one field

Bin: one bucket of values

Cross: a pair of fields

Hit count, not cycle count

Six of eight cross bins

1. You sample opcode and direction on each transaction. The cross has 4 times 2 = 8 bins.

2. After the run, 6 bins have a count above 0 and 2 bins are 0. Coverage is 6/8 = 75 percent.

3. The empty bins are opcode 3 with direction 1, and opcode 2 with direction 0. The next test is written to hit those 2 pairs, not to add random cycles.

Problem 1. All 1000 cycles use opcode 0. The coverpoint has 4 opcode bins. How many bins are empty?

Answer. 3. Only the bin for opcode 0 is hit. Cycle count does not fill the other bins.

Problem 2. A cross has 8 bins and 6 were hit. What coverage do you report, and what is the next action?

Answer. 75 percent, because 6/8 = 0.75. The next action is a constraint or a directed test for the 2 empty bins, not a longer run of the same stimulus.

Lecture 6 · 90 minutes

A scoreboard and a model

You check a stream of transactions with a scoreboard, not with a glance at a waveform. The monitor reconstructs each transaction from pins. A reference model, often a short program in a software language, predicts the result. The scoreboard compares prediction and observation. If the model says the sum of 9 and 6 is 15 and the pins say 14, you have a fail at that transaction, even if later sums look fine.

You decide what the scoreboard is allowed to reorder. A design with an in-order adder must match the model transaction by transaction, so 1 mismatch fails the run. A design that may return 2 outstanding results out of order needs a match by id, not by arrival time. If you sort neither side, you will file a bug that is really your checker. Write the matching rule down before you run the 100th random test.

The model is the specification you can execute. The scoreboard is the compare.

Monitor a transaction -> Model predicts the result -> Match by the rule -> Mismatch fails the run

Nine plus six comes back fourteen

1. The monitor sees operands 9 and 6 and a result of 14, tagged with one id.

2. The reference model computes 15. The scoreboard sees a difference of 1, so this transaction fails.

3. There is no second result in flight, so matching by arrival or by id agrees. A legal reorder of a second id would be a different case, and you would match on the id.

Problem 1. The model returns 15 for inputs 9 and 6. The pins return 14. What does the scoreboard report?

Answer. A failure on that transaction. The difference is 1. Later correct sums do not erase it.

Problem 2. Two results may complete out of order, distinguished by id. You compare by arrival time and flag a mismatch. What should you check first?

Answer. The matching rule. A legal reorder of 2 outstanding results fails an arrival-time compare even when each id matches the model. Match by id before you call the design wrong.

Lecture 7 · 90 minutes

Simulation and a bounded proof

You use simulation to run many cycles of a whole block, and you use a formal tool when you want every behavior inside a bound. A bounded check of 20 cycles searches inputs out to length 20. If an assertion can fail, the tool returns a counterexample waveform. If it cannot fail inside 20 cycles, you have a proof only up to that bound, not a proof for cycle 21.

You do not point a formal solver at an unconstrained 32-bit datapath and wait. You cut the input space with the same properties you already wrote, and you often prove control, such as a 4-entry credit counter never going negative, rather than the arithmetic. Simulation with a seed still owns the long trace, the software reference model, and the coverage of the 8 cross bins. Formal owns the claim that no short trace escapes the assertion.

Simulation samples traces. A bounded proof talks about all traces up to the bound, and no further.

Simulation

One seed, many cycles

Scoreboard and bins

Misses what you did not draw

Bound of 20

All inputs to cycle 20

A waveform if it fails

Silent about cycle 21

A counterexample of length 12

1. You ask whether a request can go ungranted for 3 cycles, inside a bound of 20.

2. The tool returns a counterexample of length 12: request at cycle 9, no grant through cycle 12. That is a bug or a missing assumption.

3. If instead it proves the property through cycle 20, a bug at cycle 21 is still possible. You say the bound in the same sentence as the proof.

Problem 1. A property is proved for a bound of 20 cycles. Have you proved it for cycle 21?

Answer. No. The search stopped at 20. Cycle 21 was not included. You may still simulate past the bound.

Problem 2. Why prove a 4-entry credit counter rather than an unconstrained 32-bit datapath?

Answer. The counter has a small input cone, so a bound of 20 can finish. An unconstrained 32-bit datapath is a search you will not exhaust. Simulation still owns long traces and the 8 coverage bins.

Lecture 8 · 90 minutes

A plan you can close

You can now write a verification plan for a small block and carry it out. You name the behaviors, drive them with directed vectors or with constrained random draws, and check them with assertions plus a scoreboard against a reference model. For a coverpoint of 4 bins you can say how many are empty. For a bound of 20 cycles you can say what a formal pass does not mean.

You can also tell a passing log from a finished plan. A run of 1000 cycles that hits 6 of 8 cross bins is not closed, and a green scoreboard does not fill the other 2 bins. You add a constraint or a directed test aimed at the hole, or you record a waiver with a reason. You can defend that choice to someone who did not watch the waveform.

Closure is a list of bins and checkers, not a cycle count.

1. Plan the behaviors

2. Choose the stimulus

3. Name the checker

4. List bins still open

Six bins and a late grant

1. A cross of 4 opcodes and 2 directions has 8 bins. The random run hits 6 and the scoreboard stays green.

2. The 2 empty bins are still open. More cycles under the same constraints do nothing if those pairs have probability 0.

3. Separately, a request granted only at cycle t+3 fails the 3-cycle property even if the scoreboard liked the payload. You fix or waive each hole on its own.

Problem 1. A cross has 8 bins and a random run hits 6. The scoreboard passed. Why might a longer run with the same constraints never finish the plan?

Answer. The 2 empty bins stay empty if the constraints give them probability 0. A green scoreboard only checked the transactions you actually drew. You change a constraint, add a directed test, or waive a bin with a reason.

Problem 2. Inputs 9 and 6 return 14, and in that same cycle 2 of 4 grant bits are 1. The model sum is 15, and the grant must be one-hot. What do you report?

Answer. Two failures. The scoreboard expected 15, so the result is wrong by 1. The immediate assertion expected at most one of the 4 grant bits. Fixing the sum does not fix the grant.